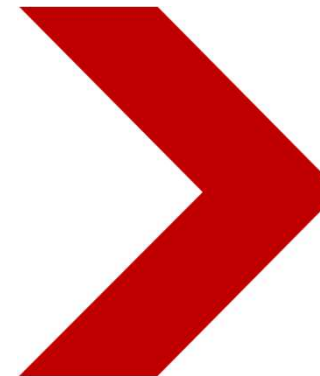


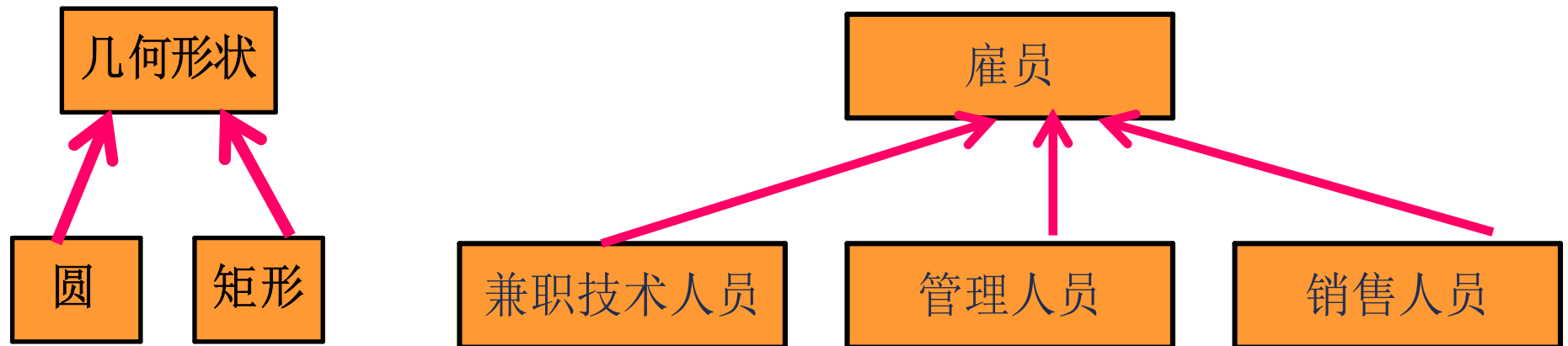
Java程序设计基础

Java Programming Fundamentals

Inheritance



Relationship of Objects



Example: Person → Student

ⓐ Person
▪ name: String ▪ age: int
● setName(name: String): void ● setAge(age: int): void ● getName(): String ● getAge(): int

ⓐ Student
▪ name: String ▪ age: int ▪ school: String
● setName(name: String): void ● setAge(age: int): void ● getName(): String ● getAge(): int ● setSchool(school: String): void ● getSchool(): String

- Person与Student存在相同属性
- Student is a Person
- 可否复用属性? → 使用继承

Inheritance

- **Reuse the existing code**
 - derive the new class from the existing class.
- **Subclass(derived class, extended class, or child class)**
 - A class that is derived from another class is called a subclass
- **Superclass (base class or parent class).**
 - The class from which the subclass is derived is called a superclass



类的继承格式

在Java中使用extends关键字完成类的继承关系，操作格式：

```
class 父类{}           // 定义父类
```

```
class 子类 extends 父类{} // 使用extends关键字实现继承
```

单独的这个类称为父类，**基类或者超类**；这多个类可以称为子类或者**派生类**。
有了继承以后，我们定义一个类时，可以在一个已经存在的类的基础上，还可扩展父类的功能。

Student extends Person

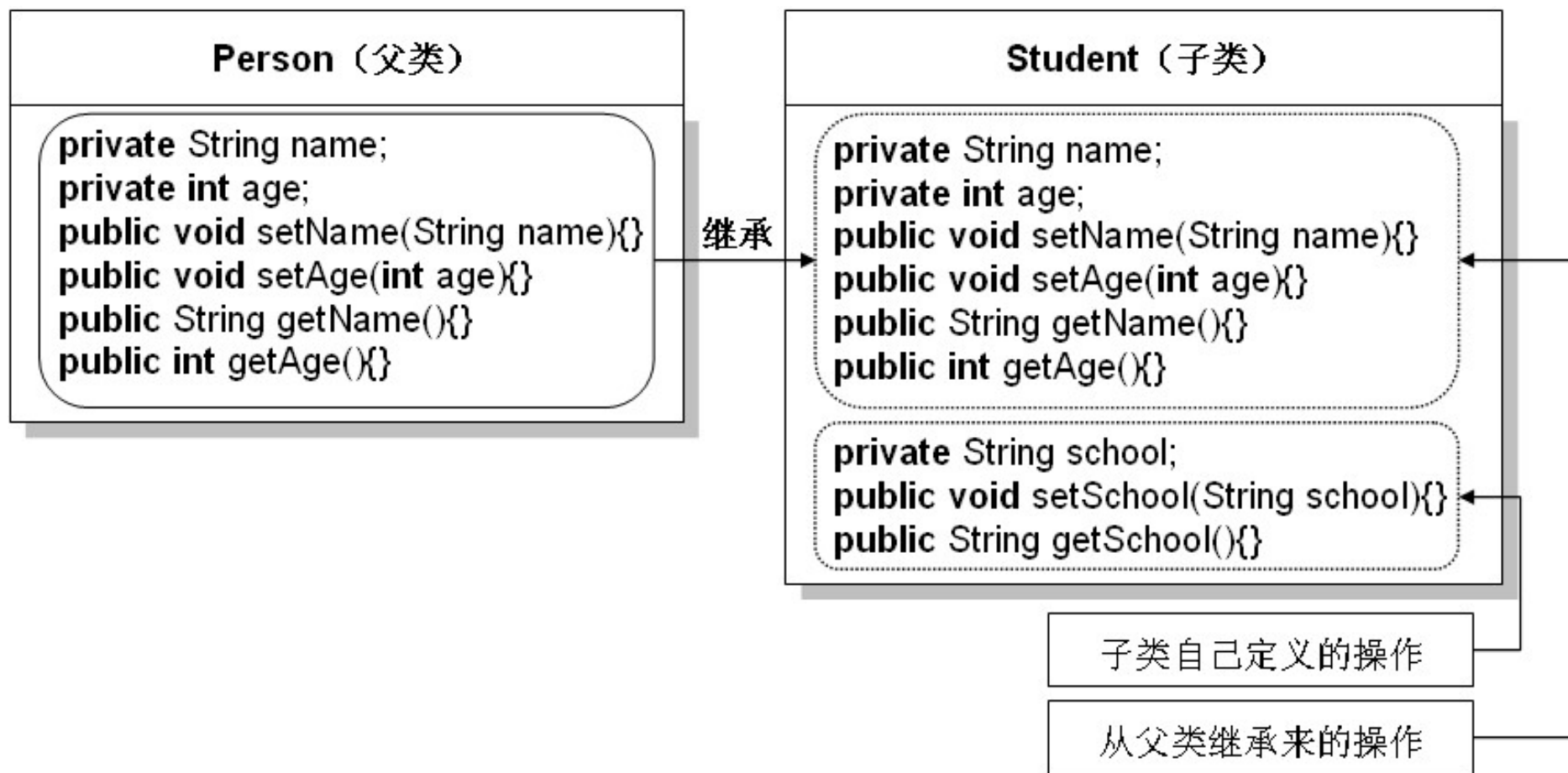
```
class Person {                                // 定义Person类
    private String name;                      // 定义name属性
    private int age;                          // 定义age属性
    public String getName() {return name; } // 取得name属性
    public void setName(String name) {      // 设置name属性
        this.name = name;
    }
    public int getAge() { return age; } // 取得age属性
    public void setAge(int age) {          // 设置age属性
        this.age = age;
    }
}

class Student extends Person {                // Student是Person的子类
    // 此处任何代码都不编写
}
```

子类扩展父类的功能

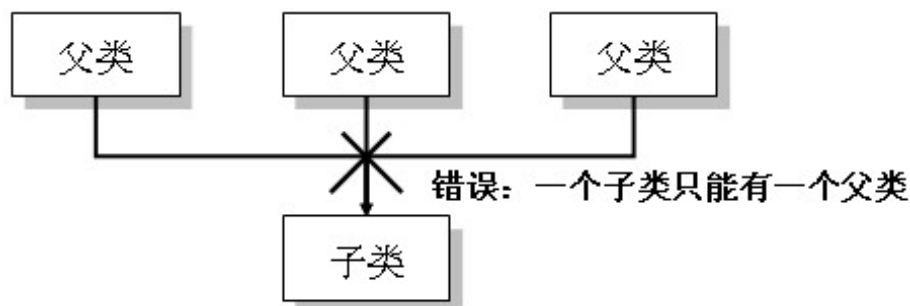
```
class Person {                                // 定义Person类
    .....//省略定义
}
class Student extends Person {                // Student是Person的子类
    private String school;                    // 新定义的属性school
    public String getSchool() {               // 取得school属性
        return school;
    }
    public void setSchool(String school) {    // 设置school属性
        this.school = school;
    }
}
```

Person与Student的继承关系图



Java不支持多重继承

在Java中只允许**单继承**，不能使用**多重继承**，
即：一个子类只能继承一个父类。但是允许进行多层继承，即：一个子类可以有一个父类，一个父类还可以有一个父类。



多重继承



多层继承

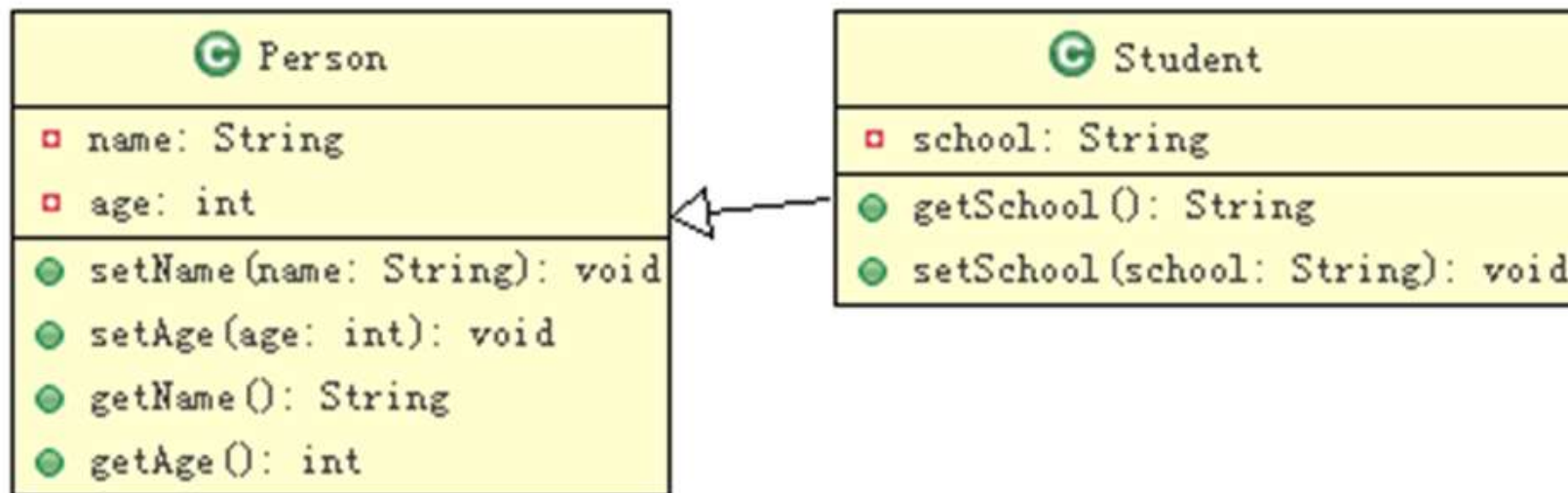
错误语法: `class 子类 extends 父类1,父类2{`



单继承	<pre>graph BT; B[Class B] --> A[Class A]</pre>	<pre>public class A { } public class B extends A { }</pre>
多重继承	<pre>graph BT; C[Class C] --> B[Class B]; B --> A[Class A]</pre>	<pre>public class A {} public class B extends A {.....} public class C extends B {.....}</pre>
不同类继承同一个类	<pre>graph BT; B[Class B] --> A[Class A]; C[Class C] --> A</pre>	<pre>public class A {} public class B extends A {.....} public class C extends A {.....}</pre>
多继承 (不支持)	<pre>graph BT; C[Class C] --> A[Class A]; C --> B[Class B]</pre>	<pre>public class A {} public class B {.....} public class C extends A,B { } // Java 不支持多继承</pre>

继承的访问限制

- 子类是不能直接访问父类中的私有成员
- 子类可以调用父类中的非私有方法

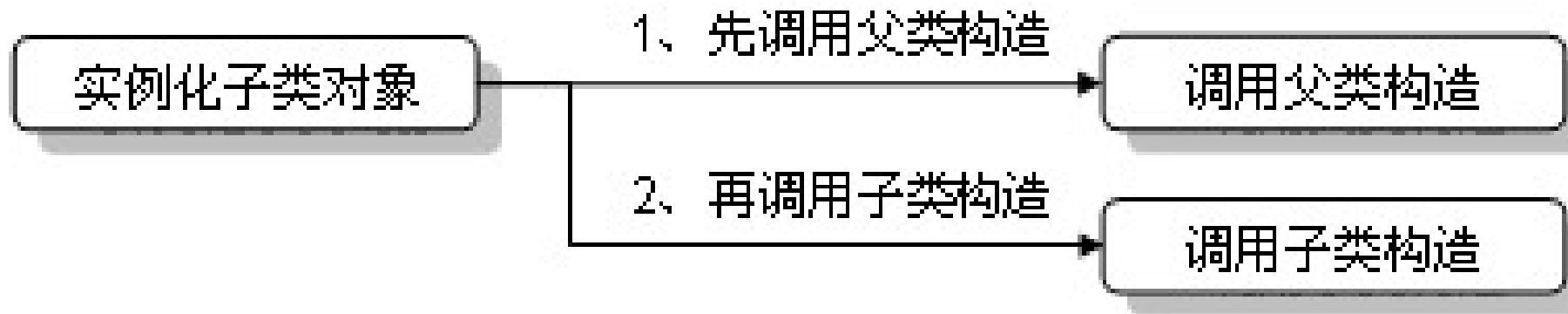


继承的访问限制

```
class Student extends Person {           // Student是Person的子类
    public void fun() {
        System.out.println("父类中的name属性: " + name) ;// 错误, 无法访问
        System.out.println("父类中的age属性: " + age) ;   // 错误, 无法访问
    }
}
```

子类的实例化

- 子类对象在实例化之前首先调用父类中的构造方法，再调用子类自己的构造方法



子类的实例化

```
class Person {                                // 定义父类Person
    private String name;                       // 定义name属性
    private int age;                           // 定义age属性
    public Person() {
        System.out.println("父类Person中的构造。") ;// 父类的构造方法
    }
    setter、getter...
}
class Student extends Person {                // Student是Person的子类，扩展父类的功能
    private String school;                     // 新定义的属性school
    public Student() {
        System.out.println("子类Student中的构造。") ;// 子类的构造方法
    }
    setter、getter...
}
public class InstanceDemo {
    public static void main(String args[]) {
        Student stu = new Student();
    }
}
```

super()的隐含调用

```
class Student extends Person {    // Student是Person的子类，扩展父类的功能
    private String school;        // 新定义的属性school
    public Student() {
        super() ;                // 加与不加此语句效果一致
        System.out.println("子类Student中的构造。") ;    // 子类的构造方法
    }
    public String getSchool() {    // 取得school属性
        return school;
    }
    public void setSchool(String school) {    // 设置school属性
        this.school = school;
    }
}
```

必须显式调用的情形

- 父类只提供带参数构造方法是，子类必须显式调用

```
class Father{
    public Father(String s) {
        System.out.println("父亲的带参构造方法");
    }
}
class Son extends Father{
    public Son() {
        super("test"); //必须是第一条语句
        System.out.println("儿子的无参构造方法");
    }
    public Son(String s) {
        super("test");
        System.out.println("儿子的有参构造方法");
    }
}
```

```
public class ExtendsDemo3 {
    public static void
    main(String[] args) {
        Son s1=new Son();
        Son s2=new Son("test");
    }
}
```


继承机制的使用原则

- 在概念上存在 **is a** 关系
- 不要因为偶然有相同功能而使用继承

```
class A{  
    public void show1(){}  
    public void show2(){}  
}  
class B{  
    public void show2(){}  
    public void show3(){}  
}
```

不推荐



```
class B extends A{  
    public void show3(){}  
}
```

虽然类B少写了一个方法show2(), 但继承A后多了一个方法show1()。



overriding—覆写

- 子类定义了与父类相同的方法（方法名称、参数类型、返回值类型全相同）
- 在方法覆写时必须考虑到权限
 - 被子类覆写的方法不能拥有比父类方法更加严格的访问权限。
 - 三种访问权限大小关系：**private < default < public**
 - 如果被子类覆写的方法权限比父类小，则在编译时将出现错误提示
 - 例如：如果在父类中使用public定义的方法，子类的访问权限必须是public，否则程序将无法编译。

overriding—覆写

```
class Person{
    void print() {                // 定义一个默认访问权限的方法
        System.out.println("Person --> void print() {}") ;
    }
}
class Student extends Person{ // 定义一个子类继承Person类
    public void print() {        // 覆写父类中的方法，扩大了权限
        System.out.println("Student --> void print() {}") ;
    }
}
public class OverrideDemo01 {
    public static void main(String args[]){
        new Student().print() ;    // 此处执行的是被覆写过的方法
    }
}
```

覆写后如何调用父类的同名方法

• 调用形式: `super.方法()`

```
class Person{  
    void print(){  
        System.out.println("Person --> void print(){}") ;  
    }  
}  
  
class Student extends Person{  
    public void print(){  
        super.print() ;  
        System.out.println("Student --> void print(){}") ;  
    }  
}  
  
public class OverrideDemo03 {  
    public static void main(String args[]){  
        new Student().print() ;  
    }  
}
```

// 定义一个默认访问权限的方法

// 定义一个子类继承Person类

// 覆写父类中的方法, 扩大了权限

// 调用父类中的print()方法

// 此处执行的是被覆写过的方法



思考

- 如果现在将父类的一个方法定义成private访问权限，在子类中将此方法声明为default访问权限，那么这样还叫做覆写吗？

属性覆盖(super, this)

- 在继承中子类声明了与父类同名的属性 → 属性覆盖

```
class Person{
    public String info = "Hello" ;    // 定义一个公共属性
}
class Student extends Person{        // 定义一个子类继承Person类
    public String info = "World" ;    // 此属性与父类中的属性名称一致
    public void print(){
        System.out.println("父类中的属性: " + super.info); // 访问父类中的info属性
        System.out.println("子类中的属性: " + this.info); // 访问本类中的info属性
    }
}
public class OverrideDemo05 {
    public static void main(String args[]){
        new Student().print() ;      // 调用print()方法
    }
}
```

Overloading vs. Overriding

No.	区别点	方法重载	方法覆写
1	单词	Overloading	Overriding
2	定义	方法名称相同，参数的类型或个数不同	方法名称、参数的类型、返回值类型全部相同
3		对权限没有要求	被覆写的方法不能拥有更严格的权限
4	范围	发生在一个类中	发生在继承类中

变量访问的就近原则

```
class Father{
    public int num=10;
    public Father(){
        System.out.println(num);
    }
}
class Son extends Father{
    public int num=20;
    public Son(){
        System.out.println(num);
    }
    public void show(){
        int num=30;
        System.out.println(num);
        System.out.println(this.num);
        System.out.println(super.num);
    }
}
```

```
public class ExtendsTest {
    public static void
    main(String[] args) {
        Son s1=new Son();
        s1.show();
    }
}
```

- 1.成员变量访问：就近原则
- 2.this访问本类的成员，super访问父类的成员
- 3.子类构造方法执行前默认先执行父类的无参构造方法

继承中代码块的执行优先顺序

```
class Father{
    {System.out.print("1");}
    public Father() {System.out.print("2");}
    static {System.out.print("3");}
}
class Son extends Father{
    static {System.out.print("4");}
    public Son() {System.out.print("5");}
    {System.out.print("6");}
}
public class ExtendsDemo3 {
    public static void main(String[] args) {
        Son s1=new Son();
    }
}
```

- 1.执行顺序：静态代码块>构造代码块>构造方法。
- 2.静态代码块的内容是随着类的加载而加载，静态代码块的内容会优先执行。
- 3.子类初始化之前会先进行父类的初始化。

final

- 使用final修饰的类不能有子类

```
final class A {}           // 使用final定义类，不能被继承
class B extends A {}      // 错误，不能继承使用final声明的类
```

- 使用final修饰的方法不能被子类覆写

```
class A {
    public final void print() { // 使用final声明的方法不能被覆写
        System.out.println("Hello");
    }
}
class B extends A {
    public final void print() { // 错误，不能覆写用final声明的方法
        System.out.println("MLDN");
    }
}
```

final

- 被final修饰的变量即成为常量

```
class A {  
    private final String INFO = "Soft" ;// 使用final声明的变量就是常量  
    public final void print() {  
        INFO = "HELLO" ;                // 错误，常量不可修改  
    }  
}
```

使用static final关键字联合声明的变量称为全局常量：

public static final String INFO = "SOFT" ;

例如JDK中的**Math.PI**是一个全局常量



个性化学习导引

- **Generative Learning in Multi-Agent Interactive Classroom**
- **<https://Open.maic.chat>**